

LISTA DE EXERCÍCIOS — UNIDADE II

Introdução e Modelagem

1. Paradigmas de programação (conceitual)

Explique, com suas próprias palavras:

- O que é programação procedural
- O que é programação orientada a objetos
- Quais são as principais diferenças entre esses paradigmas

Em seguida:

- Descreva como o problema “controle de um aluno (nome, notas e média)” seria resolvido em cada paradigma
 - Não implemente código — foque na **forma de pensar o problema**
-

2. Identificação de objetos em um domínio

Considere o domínio: **uma universidade**.

- Liste pelo menos 6 possíveis objetos (ex: Student, Course, Teacher)
- Para cada objeto, descreva:
 - 3 atributos
 - 2 comportamentos (métodos)

Objetivo: treinar **abstração inicial**

3. Objeto, identidade e estado

Considere a classe `Student`.

- Crie dois objetos com:
 - mesmo nome
 - mesma idade
 - **matrículas diferentes**

Explique:

- Por que são objetos distintos
 - Qual atributo representa a **identidade**
 - Quais atributos representam o **estado**
-

4. Primeira classe: `Student`

Implemente uma classe `Student` com:

- Atributos:
 - `name: string`
 - `registrationNumber: string`
 - `age: number`
- Crie 3 instâncias diferentes
- Imprima os dados no console

Objetivo:

- Diferenciar **classe vs instância**
-

5. Classe com comportamento: `Book`

Crie uma classe `Book` com:

- Atributos:
 - `title`

- `author`
 - `pages`
- **Método:**
 - `getDescription()` → retorna uma string com os dados do livro

Crie 2 livros e exiba suas descrições.

6. Classe com regra de negócio: `Product`

Crie uma classe `Product` com:

- **Atributos:**
 - `name`
 - `price`
 - `stock`
- **Métodos:**
 - `increaseStock(quantity)`
 - `decreaseStock(quantity)`

Regras:

- Não permitir estoque negativo
 - Validar entradas inválidas
-

7. Classe `BankAccount` (estado mutável)

Implemente:

- **Atributos:**
 - `owner`
 - `balance`
- **Métodos:**
 - `deposit(amount)`
 - `withdraw(amount)`
 - `getBalance()`

Regras:

- Não permitir saque maior que o saldo
 - Não permitir valores negativos
-

8. Construtor obrigatório (`Customer`)

Crie uma classe `Customer` onde:

- Os atributos:
 - `name`
 - `cpf`
 - `email`

Devem ser obrigatórios no construtor.

Teste:

- Tentar criar um cliente sem dados (não deve ser permitido)
-

9. Validação no construtor (`Course`)

Crie uma classe `Course` com:

- `name`
- `workloadHours`

Regra:

- A carga horária deve ser maior que zero
 - Caso contrário, lançar erro
-

10. Encapsulamento (`BankAccount` revisitada)

Refatore a classe `BankAccount` :

- Torne `balance` **privado**
- Remova qualquer acesso direto ao saldo

Permitir acesso apenas via métodos.

Objetivo:

- Introduzir **encapsulamento**
-

11. Controle de acesso (`StudentGrade`)

Crie uma classe:

- Atributos privados:
 - `grade1`
 - `grade2`
- Métodos:
 - `setGrade1(value)`
 - `setGrade2(value)`

Regras:

- Nota deve estar entre 0 e 10
 - Caso inválido, lançar erro
-

12. Comportamento derivado (`StudentGrade`)

Na mesma classe:

- Crie:
 - `getAverage()`
 - `isApproved()`

Regra:

- Aprovado se média ≥ 7
-

13. Responsabilidade de classe (análise)

Considere a classe:

```
class Student {
    saveToDatabase()
    sendEmail()
    calculateAverage()
    printReport()
}
```

Explique:

- Por que essa classe está com responsabilidades excessivas
 - Como dividir em classes mais adequadas
-

14. Classe `Rectangle`

Implemente:

- Atributos:
 - `width`
 - `height`
- Métodos:
 - `getArea()`
 - `getPerimeter()`

Validação:

- Valores devem ser positivos

15. Classe `Appointment`

Crie uma classe que represente uma consulta:

- Atributos:
 - `patientName`
 - `professionalName`
 - `date`
 - `time`
- Método:
 - `getSummary()`

Saída esperada:

```
Appointment: João with Dr. Maria on 10/05 at 10:00
```

16. Modelagem: biblioteca (conceitual)

Sem implementar código:

Modele um sistema de biblioteca com:

- Classes sugeridas:
 - `Book`
 - `User`
 - `Loan`

Para cada classe:

- Liste atributos
- Liste responsabilidades

17. Modelagem: restaurante

Modele um sistema de pedidos:

- Classes:
 - `Customer`
 - `Product`
 - `Order`
 - `OrderItem`

Explique:

- O papel de cada classe
- O que cada uma deve ou não fazer

18. Coesão (`ShoppingCart`)

Crie uma classe `ShoppingCart` com:

- Lista de produtos
- Método `getTotal()`

Evite:

- lógica de pagamento
- envio de email
- persistência

Objetivo:

- alta coesão
-

19. Acoplamento (análise)

Considere:

```
class Order {  
    confirm() {  
        const email = new EmailSender()  
        email.send()  
    }  
}
```

Explique:

- Por que há alto acoplamento
 - Qual problema isso pode causar
 - Como melhorar (sem usar ainda conceitos avançados)
-

20. Refatoração de modelo (sale)

Dada a estrutura:

```
class Sale {  
    customerName  
    customerCpf  
    productName  
    productPrice  
}
```

Refatore para:

- Criar classes:
 - Customer
 - Product
 - Sale

Explique:

- Como a nova modelagem melhora coesão e organização